

CPSC 340:

Machine Learning and Data Mining

Deep Learning

Deep learning → $\left[\begin{array}{l} \text{Neural net with many hidden} \\ \text{layers:} \end{array} \right]$

Last Time: Neural Networks

- Neural networks with one hidden layer:

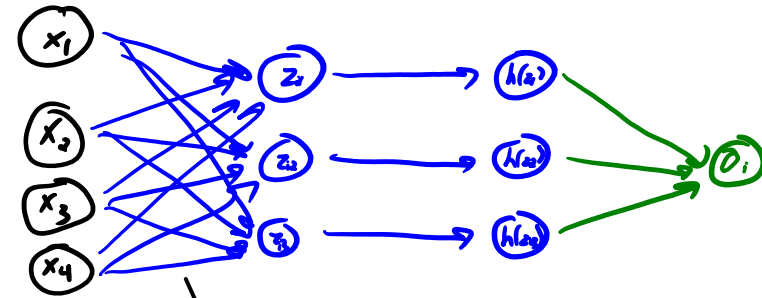
- Learn features and classifier at the same time.
- Output is two linear transformations (W, v) , separated by non-linearity (h) :

$$o_i = v^T h(W x_i)$$

Linear combination of "activations"

Non-linear transformation of each z_i

" z_i ": linear combination of input



- Linear classification/regression using non-linearly transformed latent features z_i .
- Optimize logistic/softmax loss (classification) or squared error loss (regression) using SGD:

$$\frac{1}{2} \sum_{i=1}^n (\underbrace{v^T h(W x_i)}_{o_i} - y_i)^2$$

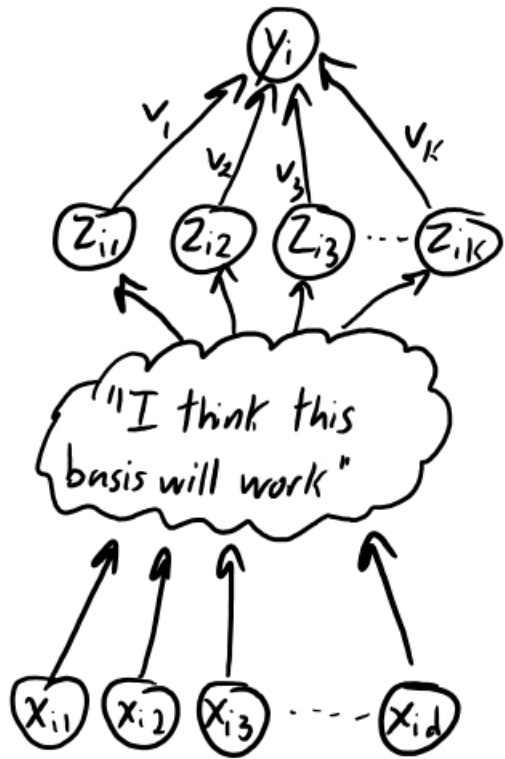
(regression)

$$\sum_{i=1}^n \log(1 + \exp(-y_i \underbrace{v^T h(W x_i)}_{o_i}))$$

(binary classification)

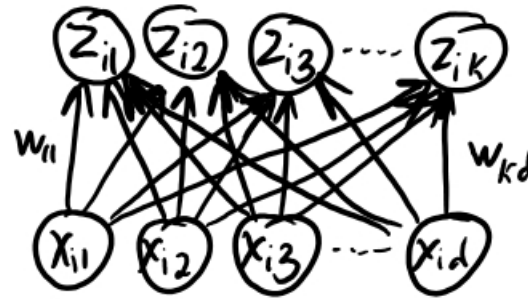
Supervised Learning Roadmap

Hand-engineered features:

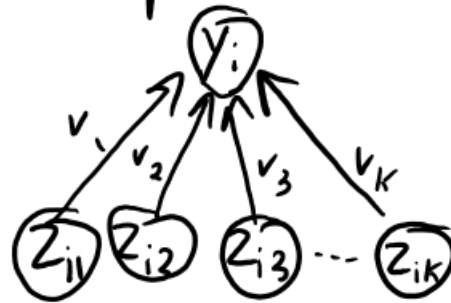


Requires domain knowledge
and can be time-consuming

Learn a latent-factor model:

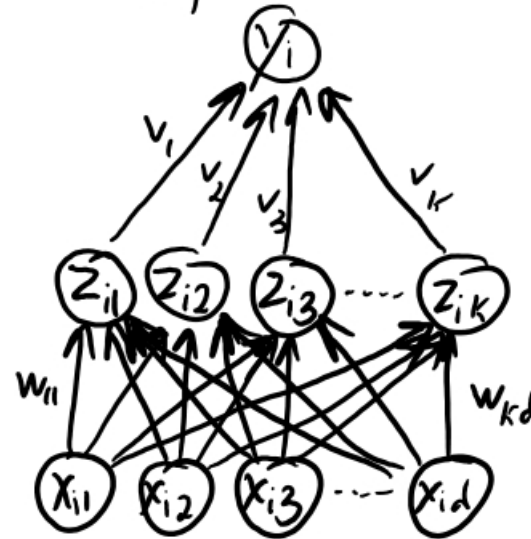


Use latent features
in supervised model:



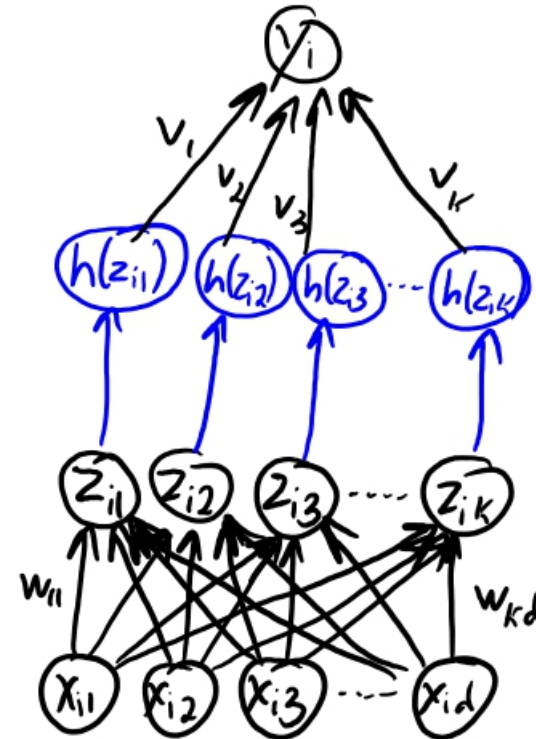
Good representation of
 x_i might be bad for predicting y_i

Learn ' v ' and ' W '
together:



But still gives a
linear model.

Neural network:

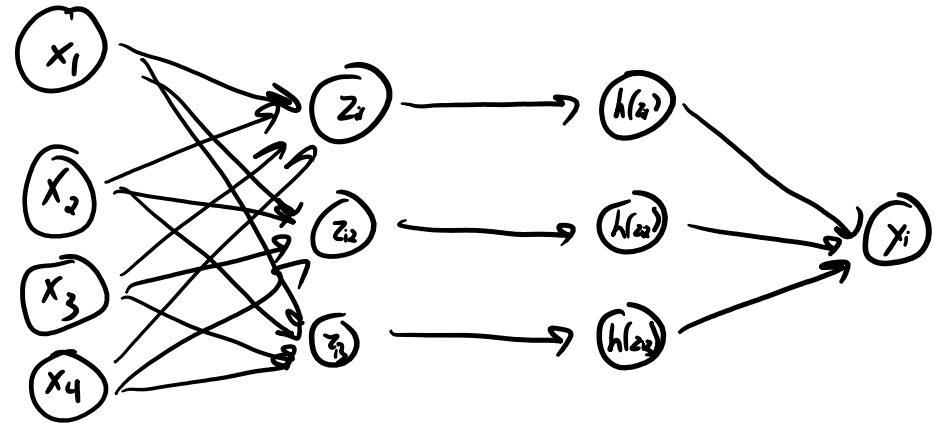


Extra non-linear
transformation ' h '

Regression vs. Binary Classification

- For **regression** problems, our prediction (ignoring biases) is:

$$\hat{y}_i = v^T h(W x_i)$$

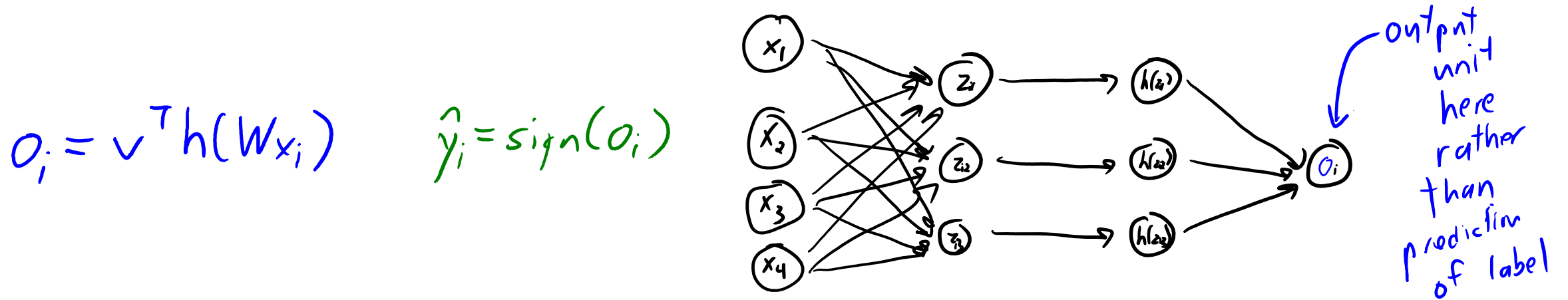


- And we might train to minimize the **squared residual**:

$$f(W, v) = \frac{1}{2} \sum_{i=1}^n (\hat{y}_i - y_i)^2 = \frac{1}{2} \sum_{i=1}^n (v^T h(W x_i) - y_i)^2$$

Regression vs. Binary Classification

- For **binary classification** problems, our prediction is:



- And we might train to minimize the **logistic loss**:

$$f(W, v) = \sum_{i=1}^n \log(1 + \exp(-y_i o_i)) = \sum_{i=1}^n \log(1 + \exp(-y_i v^T h(Wx_i)))$$

– This is like **logistic regression** with 'learned features'.

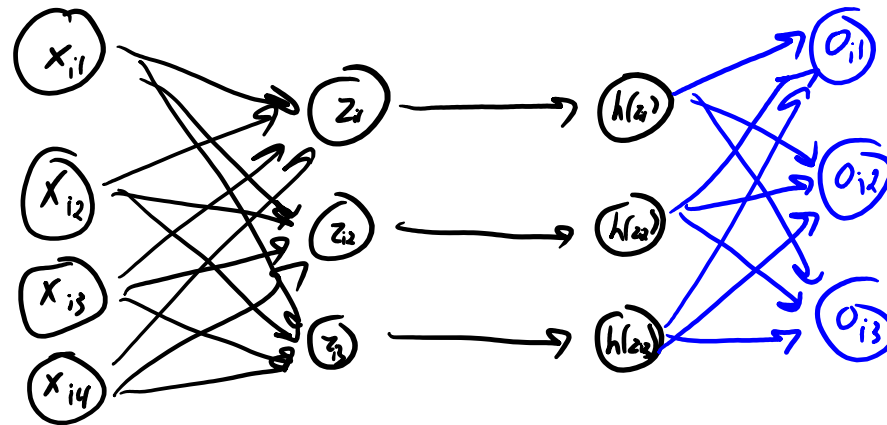
$$p(y_i | W, v, x_i) = \frac{1}{1 + \exp(-y_i \underbrace{v^T h(Wx_i)}_{o_i})}$$

Use a Sigmoid on output to get a probability

Neural Network for Multi-Class Classification

- Multi-class classification a **neural network**:

- Input is connected to a hidden layer (same as regression and binary case).
- **Hidden layer is connected to multiple output units** (one for each label.).



$$o_{i1} = v_1^T h(Wx_i)$$

$$o_{i2} = v_2^T h(Wx_i)$$

$$o_{i3} = v_3^T h(Wx_i)$$

Now have a matrix of parameters:

$$V = \begin{bmatrix} \text{---} v_1^T \text{---} \\ \text{---} v_2^T \text{---} \\ \text{---} \vdots \text{---} \\ \text{---} v_k^T \text{---} \end{bmatrix}$$

$l \times k$
 number of classes $\rightarrow$ number of hidden units

- We can predict by **maximizing** o_{ic} over all 'c'.
- We can **convert to probabilities** for each class using **softmax** to the o_{ic} values:

$$\frac{\exp(o_{ic})}{\sum_{c'=1}^{K'} \exp(o_{ic'})}$$

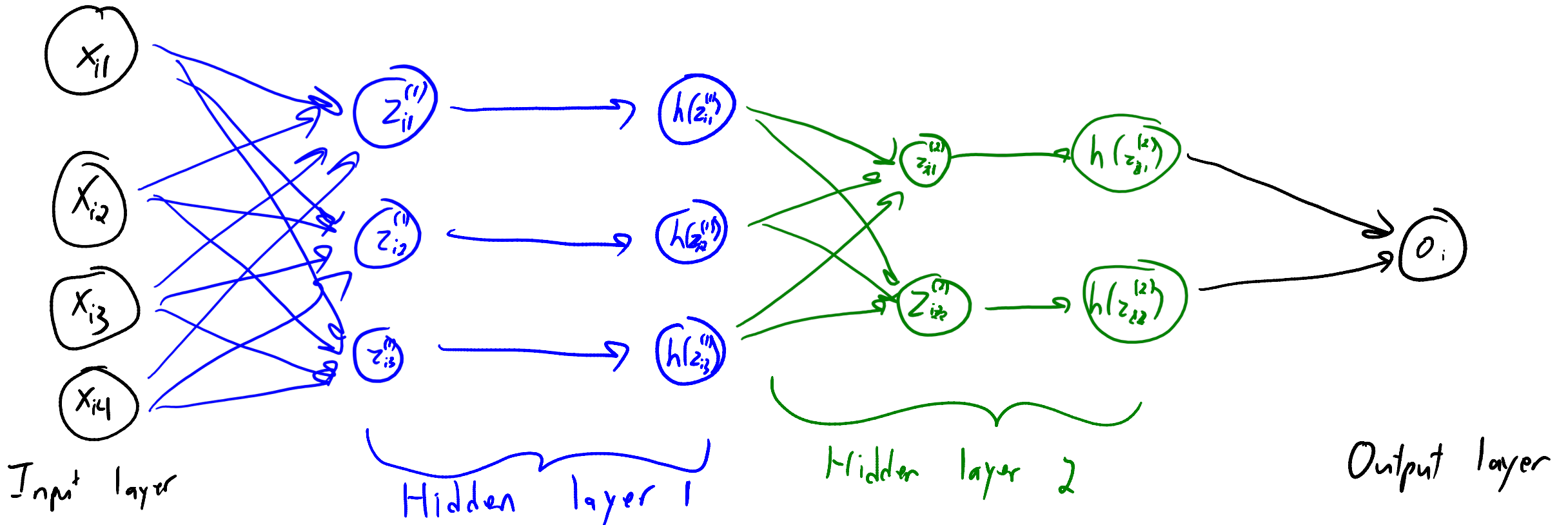
- We train by **minimizing negative log** of this probability (softmax loss, summed across examples).
- Notice that we **changed tasks by only changing last layer** (and loss function).

aka "cross entropy" (good intuitive explanation here)

Next Topic: Deep Learning

Deep Learning (As a Picture)

- Deep learning models have more than one hidden layer:



- We apply linear transformation and activation function at each "layer".

Deep Learning (As a Function)

Linear model:

$$o_i = w^T x_i$$

Neural network with 1 hidden layer:

$$o_i = v^T h(\underbrace{W x_i}_{z_i})$$

Neural network with 2 hidden layers:

$$o_i = v^T h(\underbrace{W^{(2)} h(\underbrace{W^{(1)} x_i}_{z_i^{(1)}})}_{z_i^{(2)}})$$

Neural network with 3 hidden layers

$$o_i = v^T h(\underbrace{W^{(3)} h(\underbrace{W^{(2)} h(\underbrace{W^{(1)} x_i}_{z_i^{(1)}})}_{z_i^{(2)}})}_{z_i^{(3)}})$$

Neural networks with 4 hidden layers:

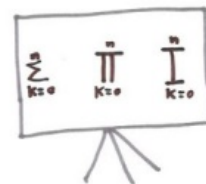
$$o_i = v^T h(W^{(4)} h(W^{(3)} h(W^{(2)} h(W^{(1)} x_i))))$$

With 'm' layers we could use:

$$o_i = v^T \left(\prod_{l=1}^m h(W^{(l)} x_i) \right)$$

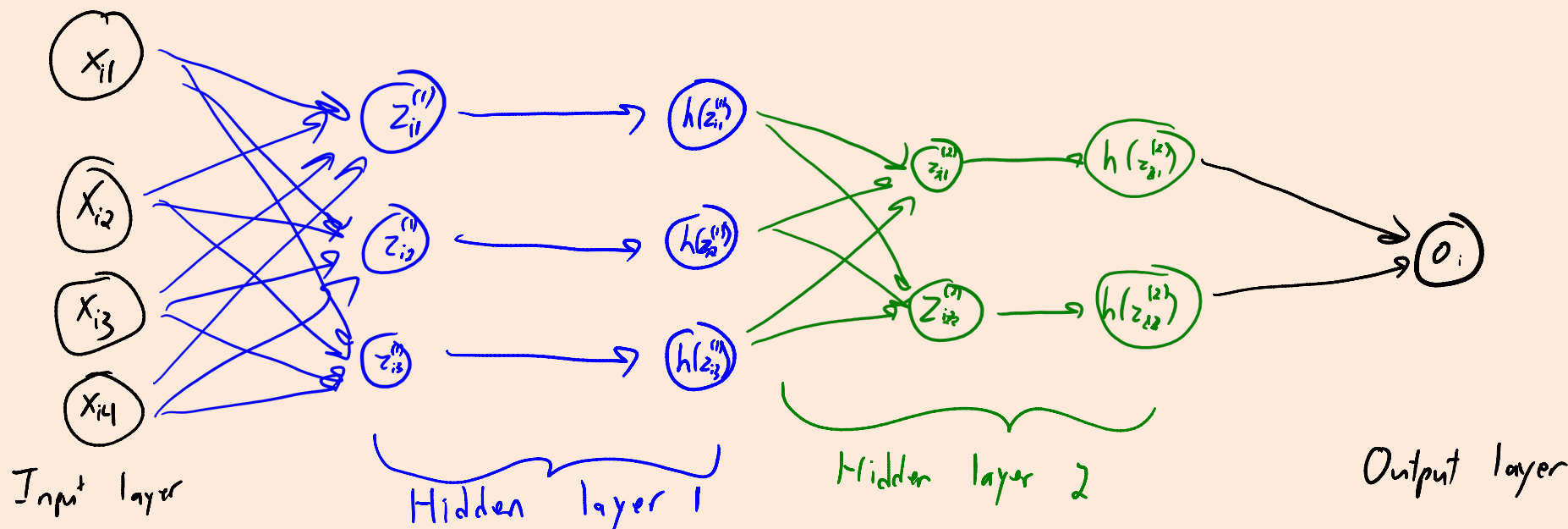
Symbol: $\prod_{k=0}^n f_k(t)$

Meaning: $f_n \circ f_{n-1} \circ f_{n-2} \dots \circ f_2 \circ f_1 \circ f_0(t)$



Notation Warning: “Number of Layers”

- In this class, we say that the network below has “2 hidden layers”:
 - Number of intermediate hidden unit groups is number of “layers”.



- Caution: exist other ways of counting the number of “layers”.
 - Some sources would refer to the above as a 3-layer neural network.
 - They count the number of linear transformations we do.
 - So network with 1 hidden layer would be a “2-layer” network, and linear models are “1-layer networks”.

Prediction with Deep Neural Networks

- The “textbook” choice for deep neural networks:
 - Alternate between doing linear transformations and non-linear transforms.

$$O_i = v^T h(W^{(4)} h(W^{(3)} h(W^{(2)} h(W^{(1)} x_i))))$$

- Each “layer” might have a different size.

- W^1 is $k^1 \times d$.
- W^2 is $k^2 \times k^1$.
- W^3 is $k^3 \times k^2$.
- W^4 is $k^4 \times k^3$.
- v is $k^4 \times 1$.

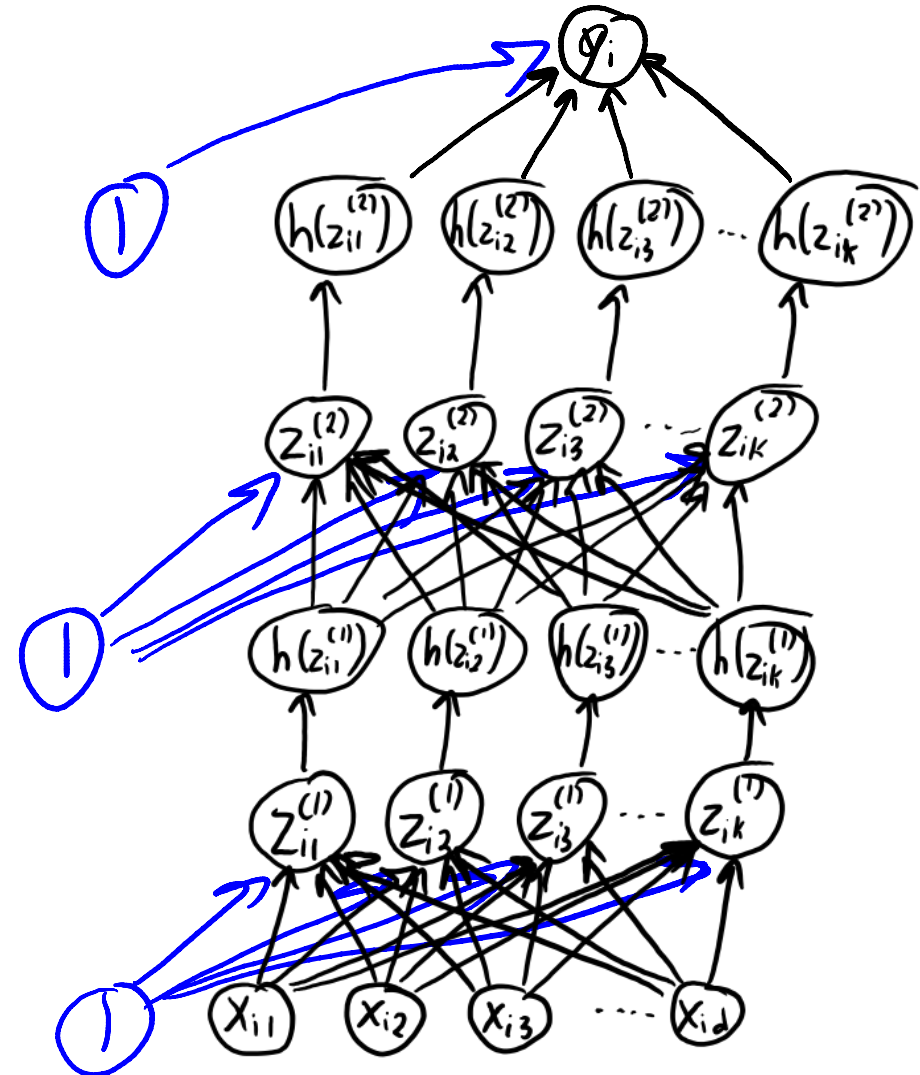
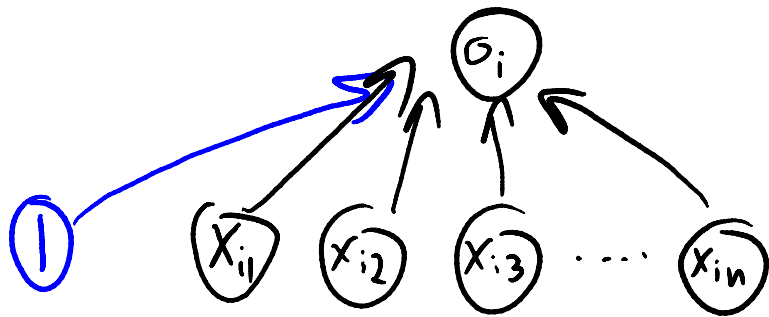
```
z[1] = W1*x
for layer in 2:nLayers
    z[layer] = Wm[layer-1]*h(z[layer-1])
end
yhat = v'*h(z[end])
```

- We may use the same non-linear transform, such as sigmoid, at each layer.
- Cost for prediction, which is called “forward propagation”:
 - Cost of the matrix multiplies: $O(k^1d + k^2k^1 + k^3k^2 + k^4k^3)$
 - Cost of the non-linear transforms is $O(k^1 + k^2 + k^3 + k^4)$, so does not change cost.
- Only need to change the last layer based on task (like regression or classification).
 - Squared error, logistic, softmax, and so on.

Adding Bias Variables

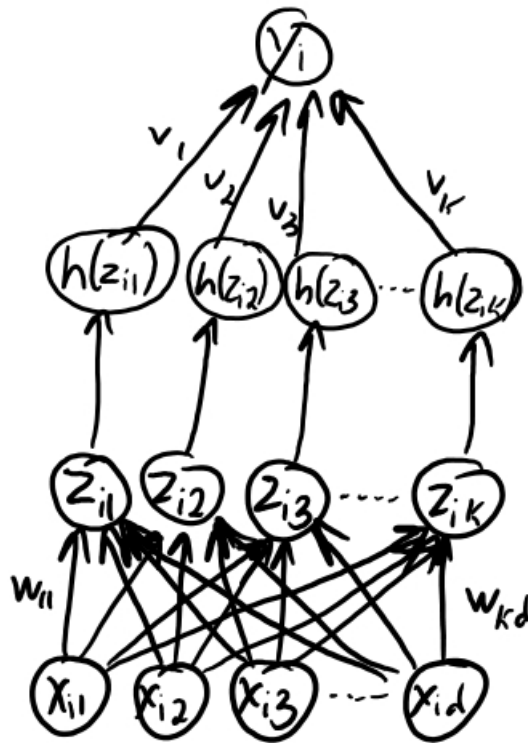
- We typically add a bias to each layer:

Linear model with bias:



Deep Learning

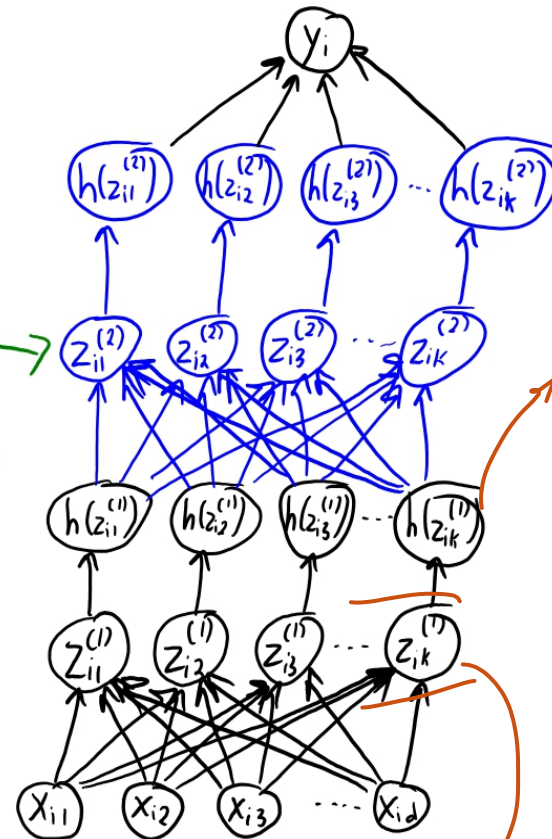
Neural network:



Deep learning:

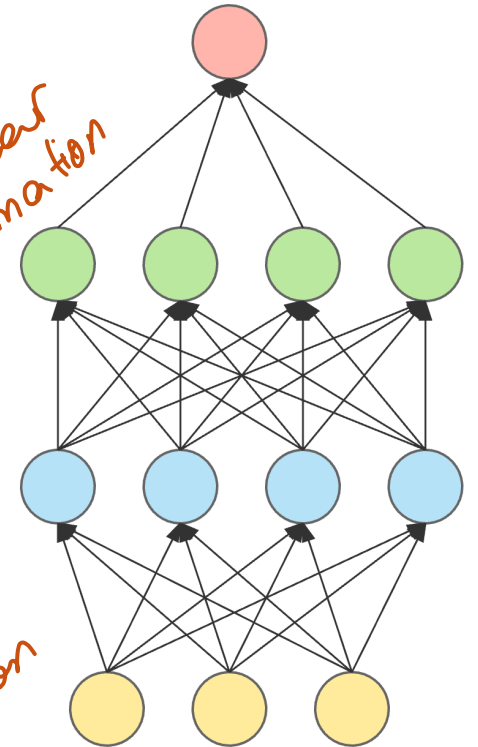
Second "layer" of latent features

You can add more "layers" to go "deeper"



non-linear transformation

linear transformation



output layer

hidden layer 2

hidden layer 1

input layer

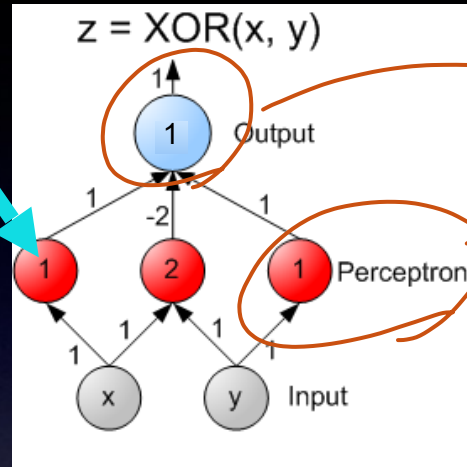
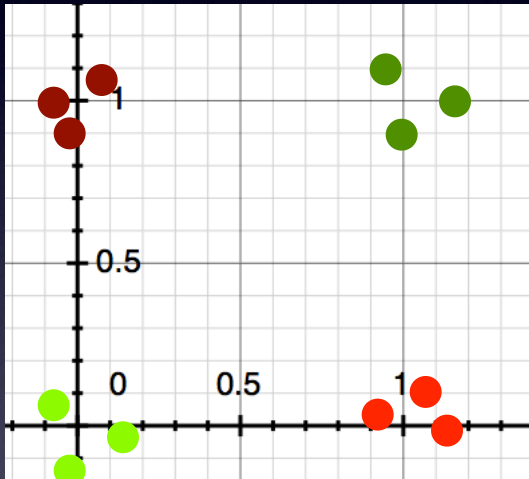
Next Topic: Solving the XOR problem


Exclusive OR

Solving the XOR Problem

Outputs 1 if $\geq$ number in node
Else: 0

2D



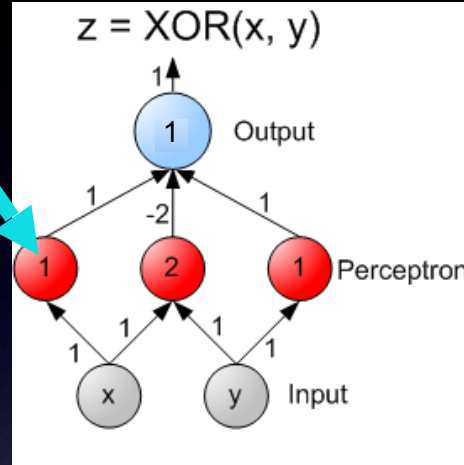
Linear Function

Cannot be done on Lm $\rightarrow$ no Line

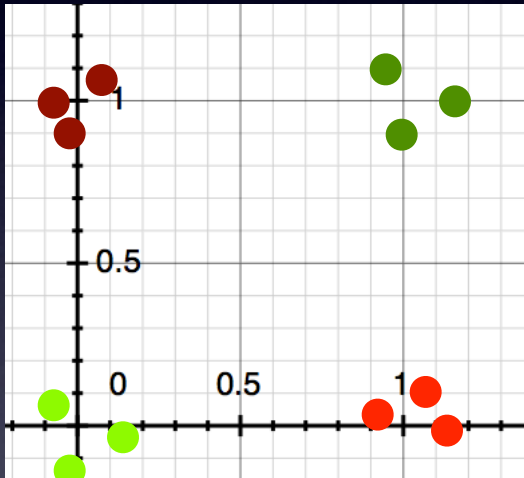
	x	y	Left	Center	Right	Output
Outputs of each node	●	●	1	1		
	●	●	1	0		
	●	●	0	1		
	●	●	0	0		

Solving the XOR Problem

Outputs 1 if $\geq$ number in node
Else: 0



2D

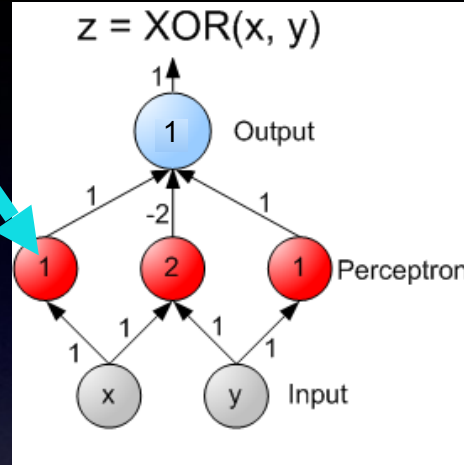


Outputs of each node

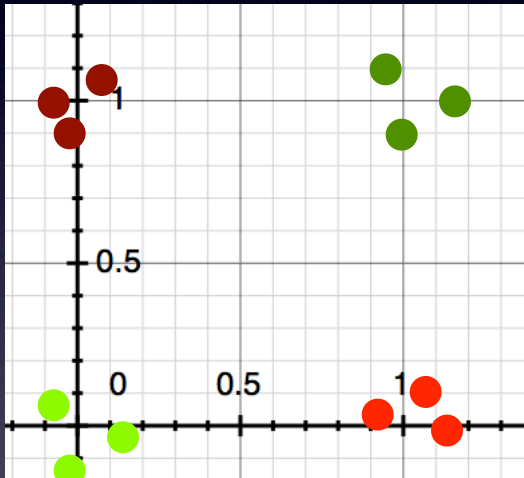
	x	y	Left	Center	Right	Output
●	1	1	1	1	1	0
●	1	0	1	1	1	1
●	0	1	1	1	1	1
●	0	0	1	1	1	0

Solving the XOR Problem

Outputs 1 if $\geq$ number in node
Else: 0



2D

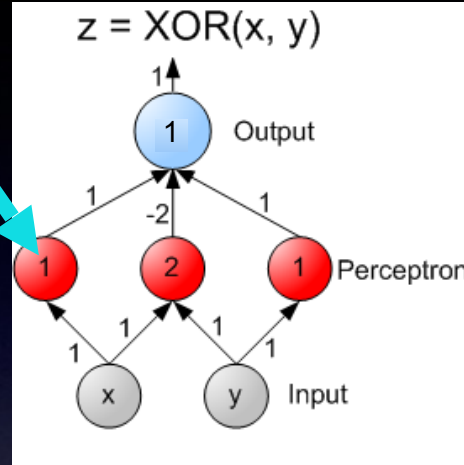


Outputs of each node

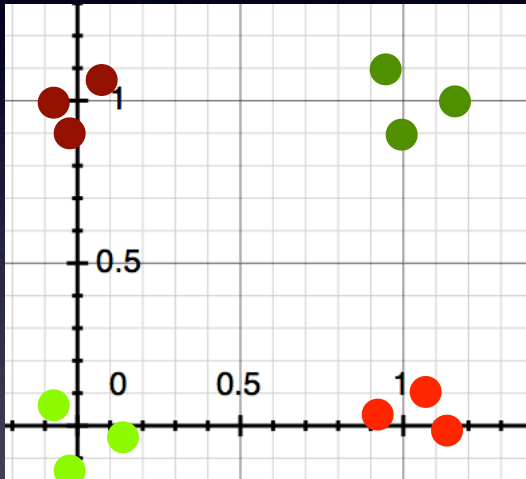
	x	y	Left	Center	Right	Output
Green	0	0	0	0	0	0
Red	0	1	1	0	0	1
Brown	1	0	1	0	0	1
Light Green	1	1	0	1	1	0

Solving the XOR Problem

Outputs 1 if $\geq$ number in node
Else: 0



2D



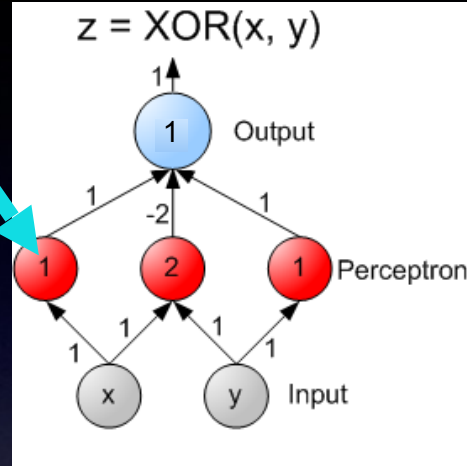
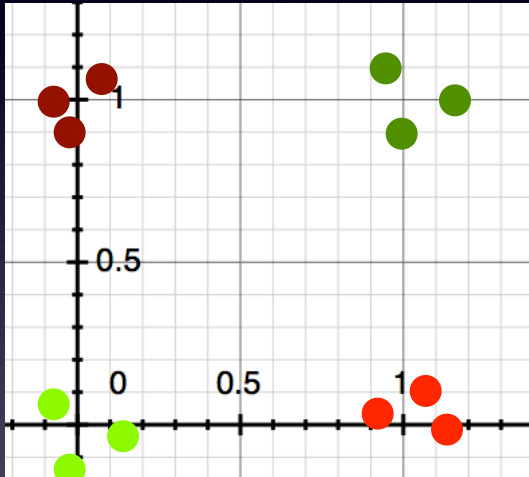
Outputs of each node

	x	y	Left	Center	Right	Output
Green	1	1	1	1	1	0
Red	1	0	1	0	0	1
Brown	0	1	0	0	1	1
Light Green	0	0	0	0	0	0

Solving the XOR Problem

Outputs 1 if $\geq$ number in node
Else: 0

2D

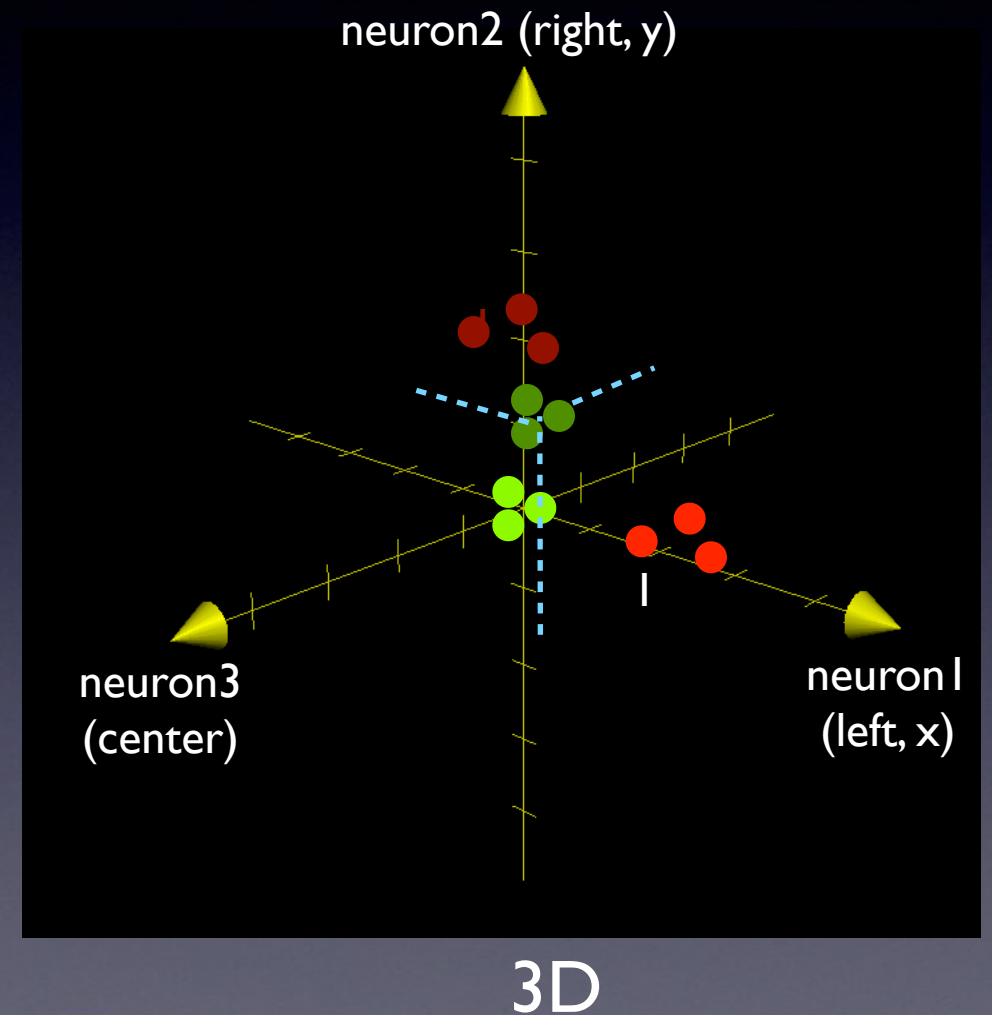


Non-Linear Mapping

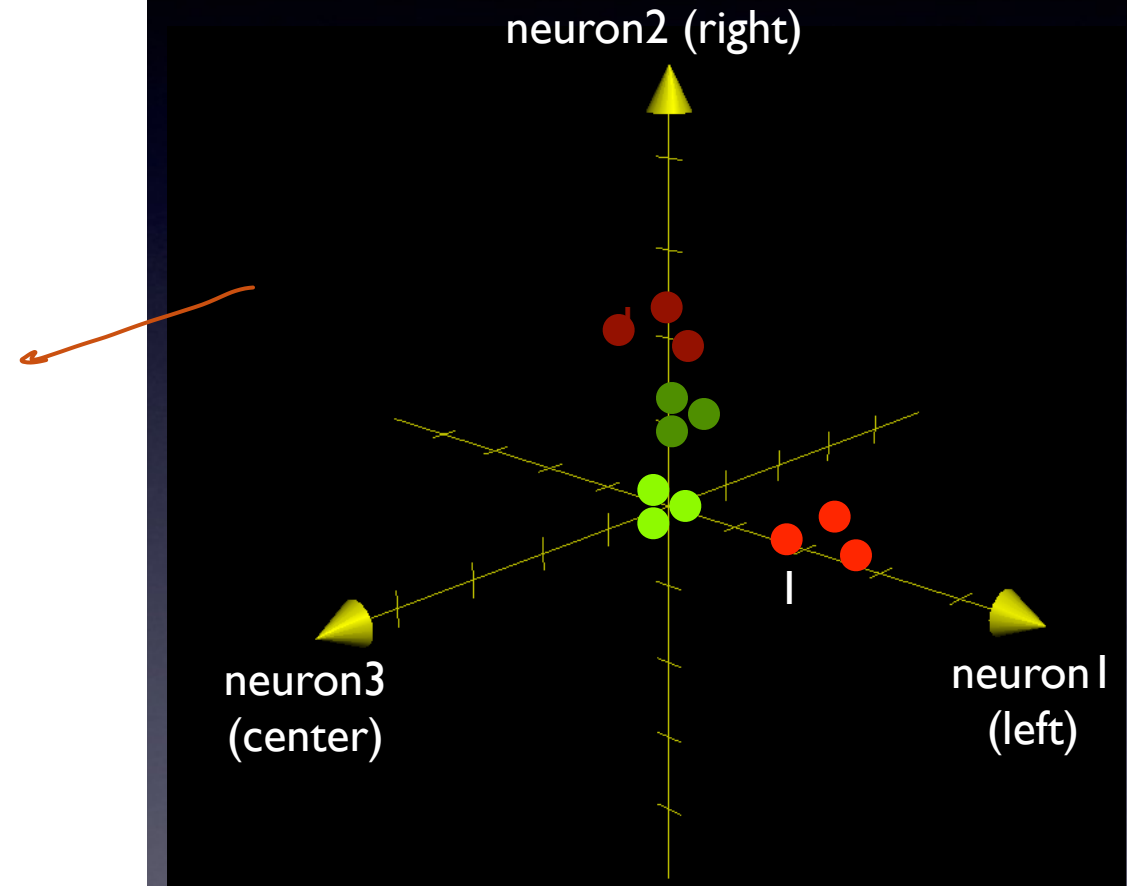
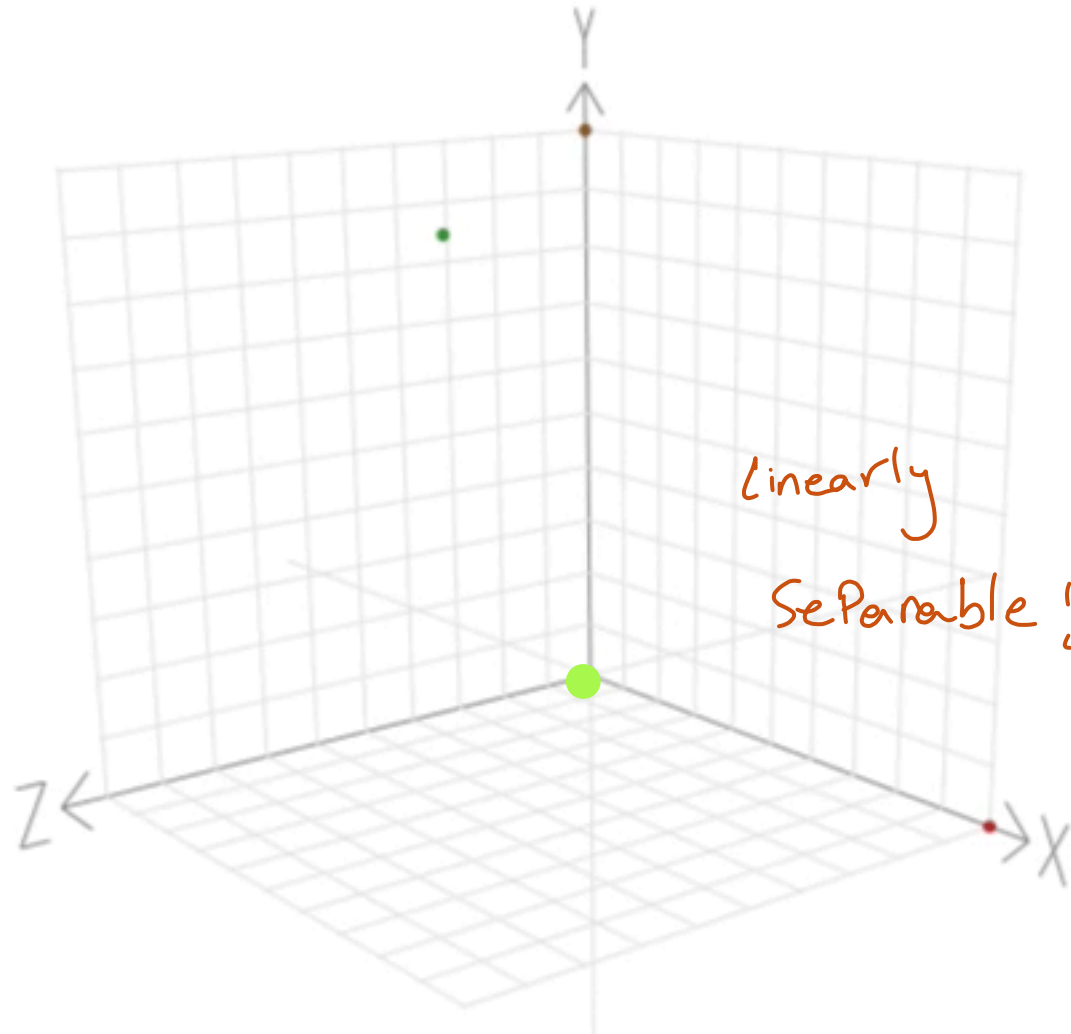
Feature Transformation

Outputs of each node

	x	y	Left	Center	Right	Output
Green	1	1	1	1	1	0
Red	1	0	1	0	0	1
Brown	0	1	0	0	1	1
Yellow	0	0	0	0	0	0



Solving the XOR Problem



3D

Solving the XOR Problem

- Point of XOR was not that you need $k > d$
 - Instead, linear transformations:
 - an example of how that can make things easier
 - and can make things linearly separable
- Cover's theorem: "The probability that classes are linearly separable increases when the features are nonlinearly mapped to a higher dimensional feature space." [Cover 1965]

The output layer requires linear separability.

جواب سے

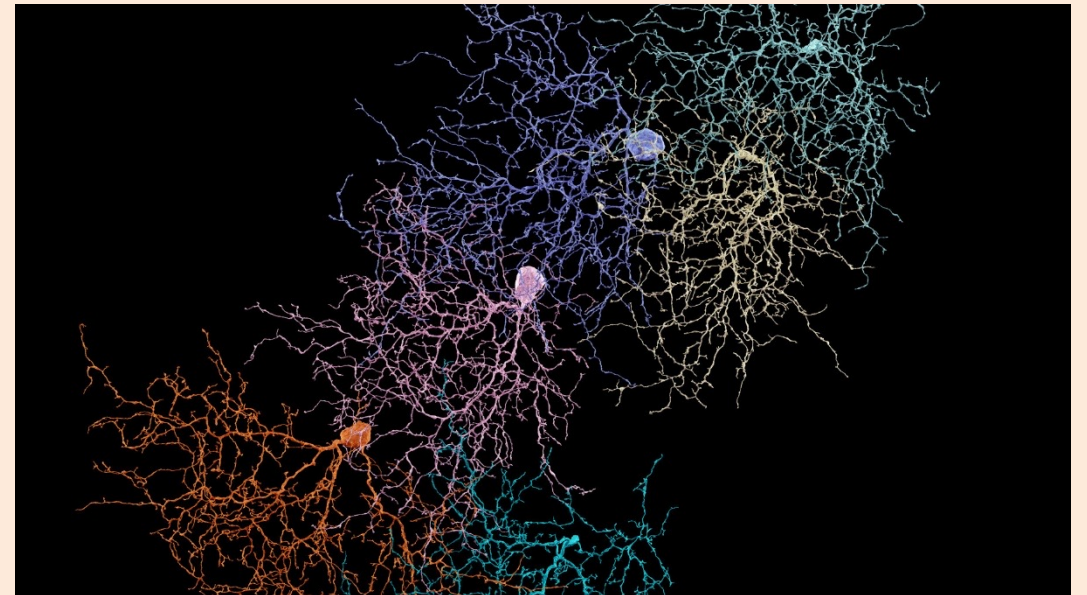
- The purpose of the hidden layers is to make the problem linearly separable!
- Multi-layer networks thus allow "non-linear regression"

Next Topic: Why Multiple Layers

Remember that
widely enough one-layer
NN is a universal approximation!
But it is more efficient
with lower depth.

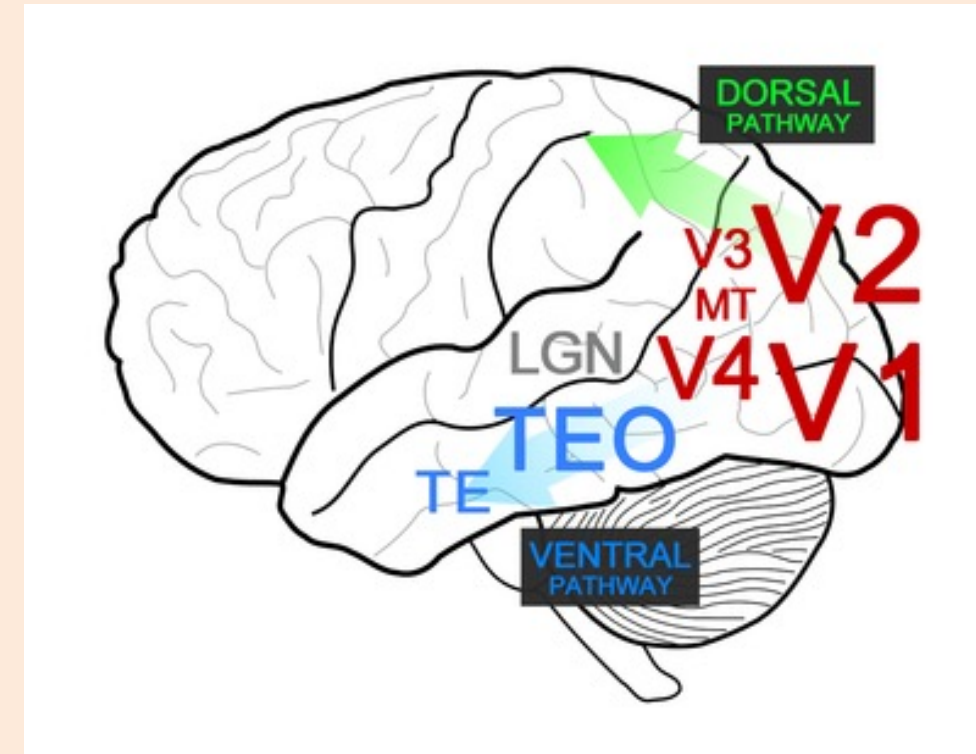
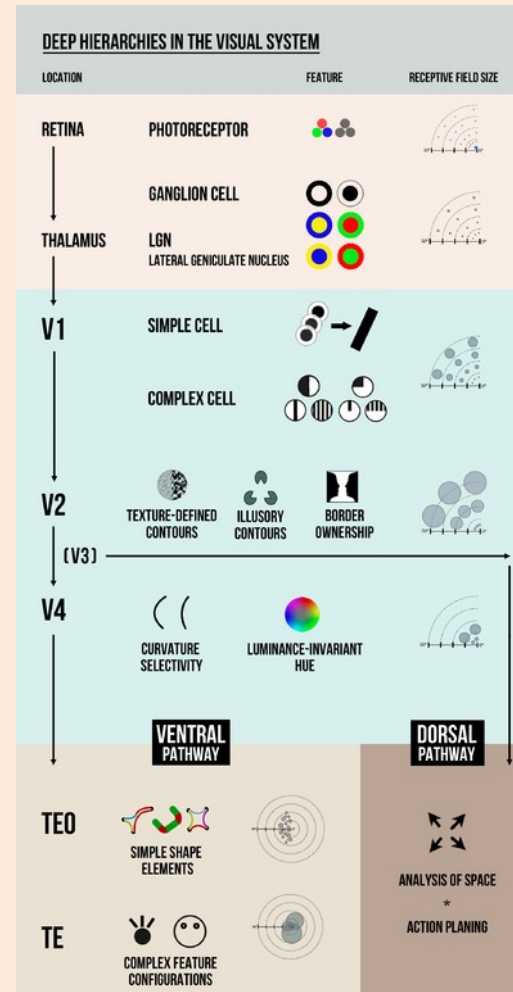
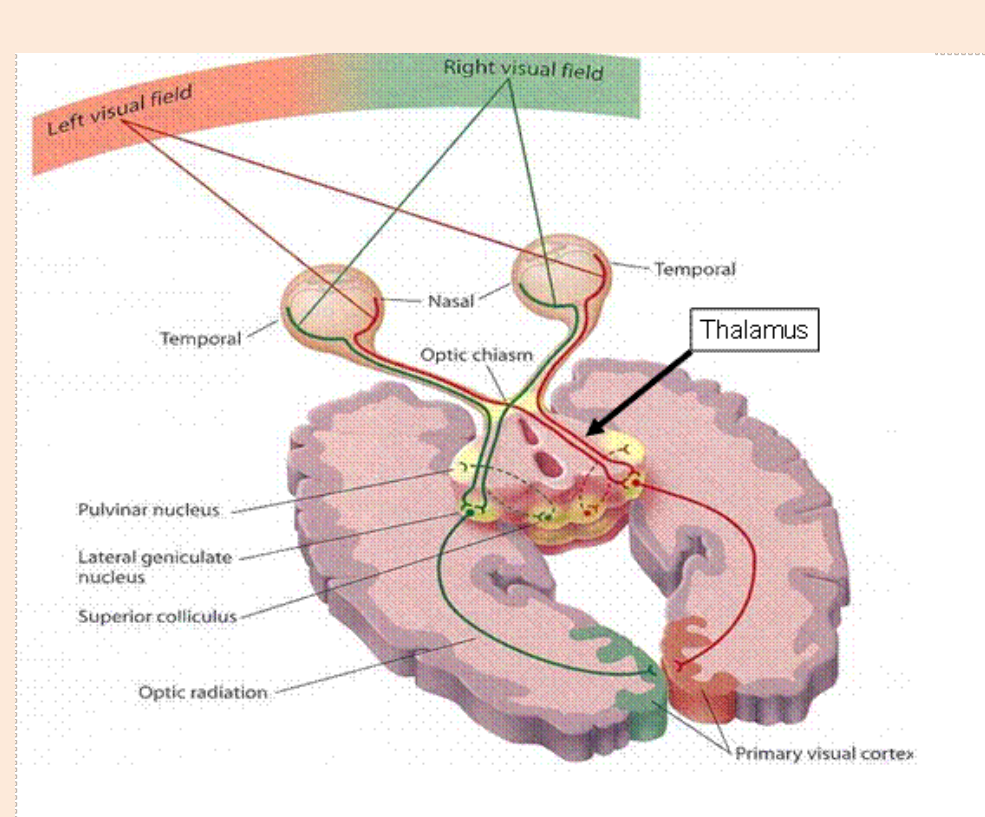
Why Multiple Layers?

- Historically, deep learning was motivated by “**connectionist**” ideas:
 - **Brain consists of network of highly-connected simple units.**
 - Same units repeated in various places.
 - Computations are done in parallel.
 - Information is stored in distributed way.
 - Learning comes from updating of connection strengths.
 - One learning algorithm used everywhere.



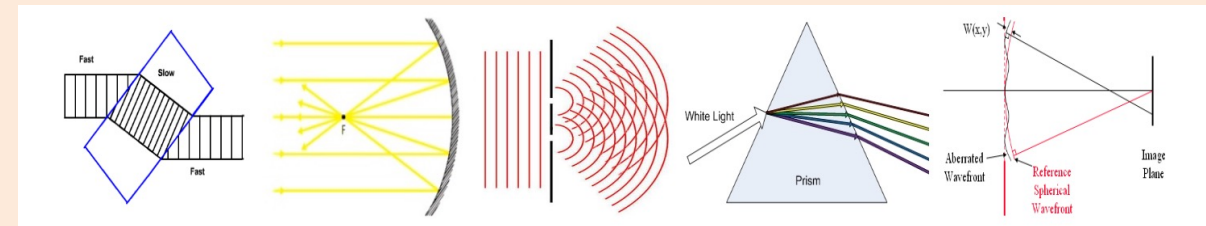
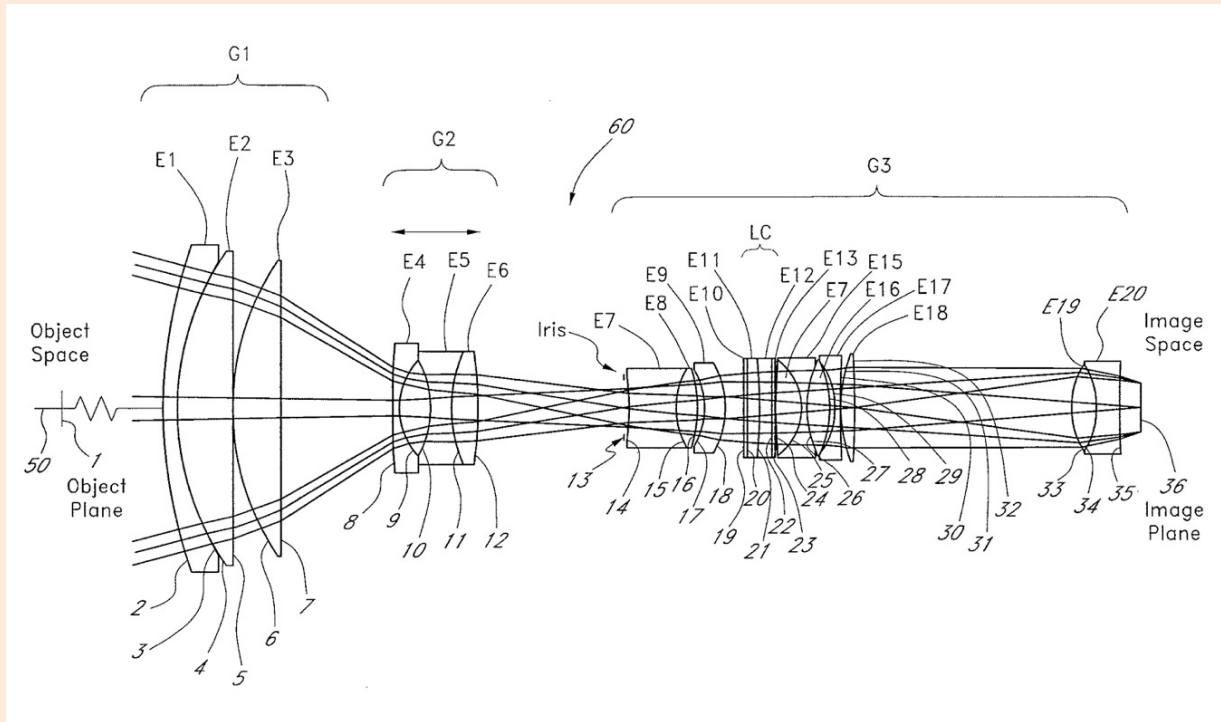
Why Multiple Layers?

- And theories on the **hierarchical organization of the visual system**:



Why Multiple Layers?

- The idea of multi-layer designs appears in engineering too:
 - Deep hierarchies in camera design:



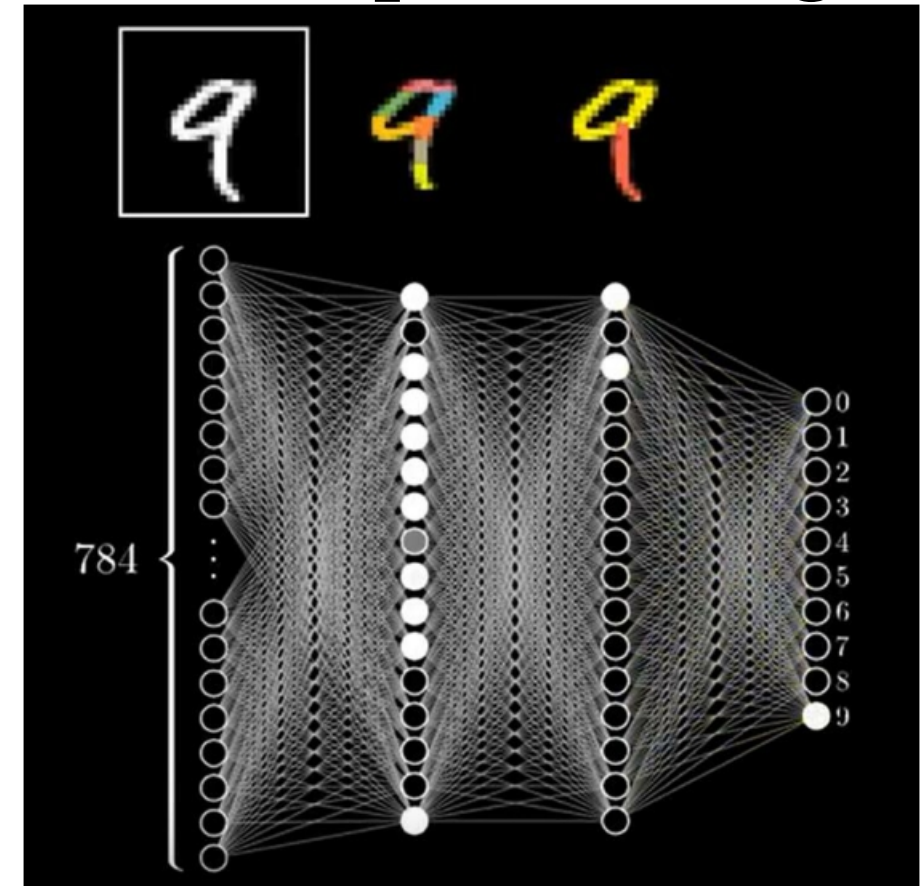
Why Multiple Layers?

- There are also **mathematical motivations** for using multiple layers:
 - 1 layer gives us a universal approximator.
 - But this layer might need to be huge.
 - With deep networks:
 - Some functions can be approximated with exponentially-fewer parameters.
 - Compared to a network with 1 hidden layer.
 - So deep networks may need fewer parameters than “shallow but wide” networks.
 - And hence may need less data to train.
- **Empirical motivation** for using multiple layers: Empirical
 - In many domains deep networks have led to unprecedented performance.

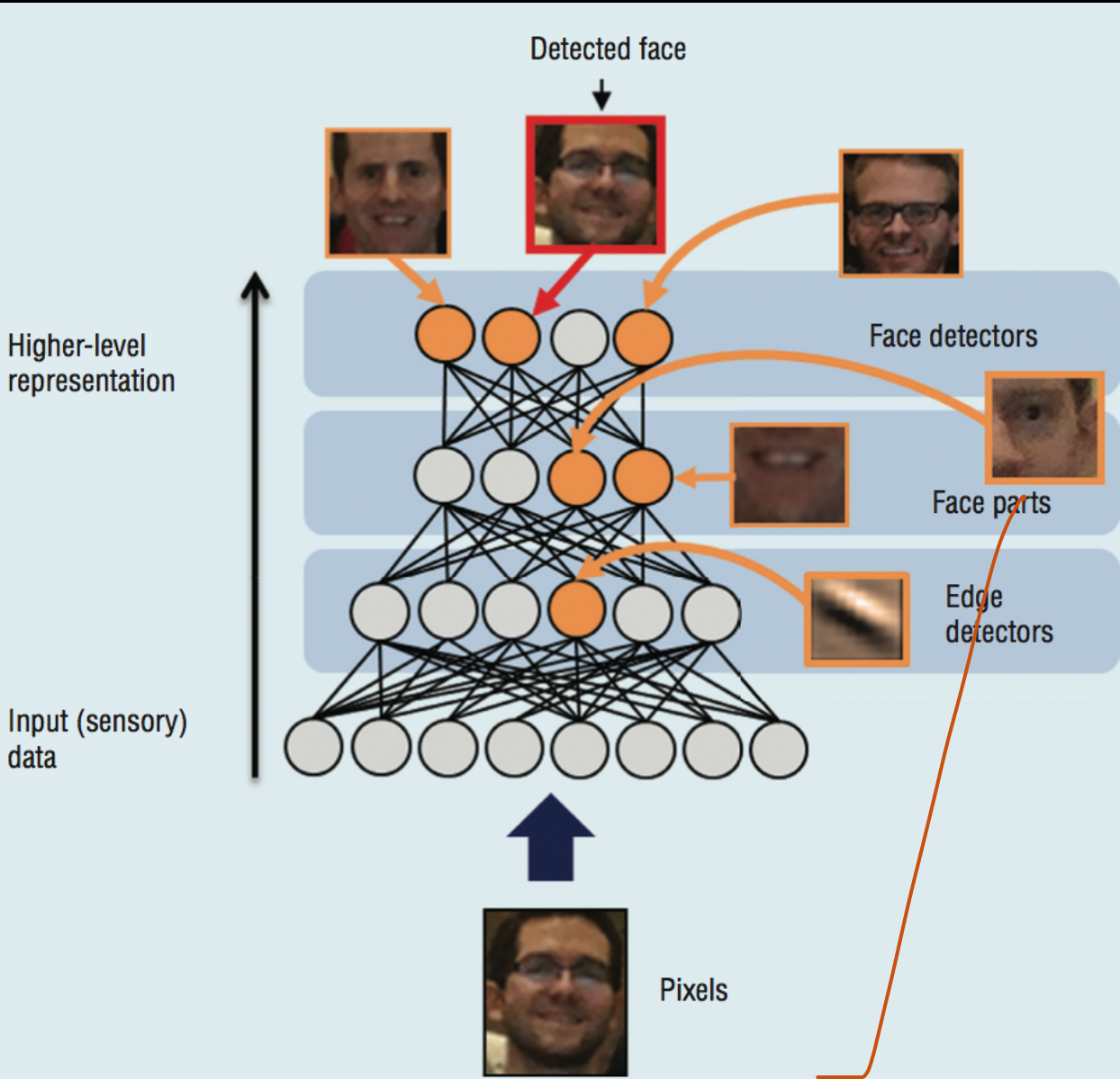
most frequently-used words now?

“Hierarchies of Parts” Motivation for Deep Learning

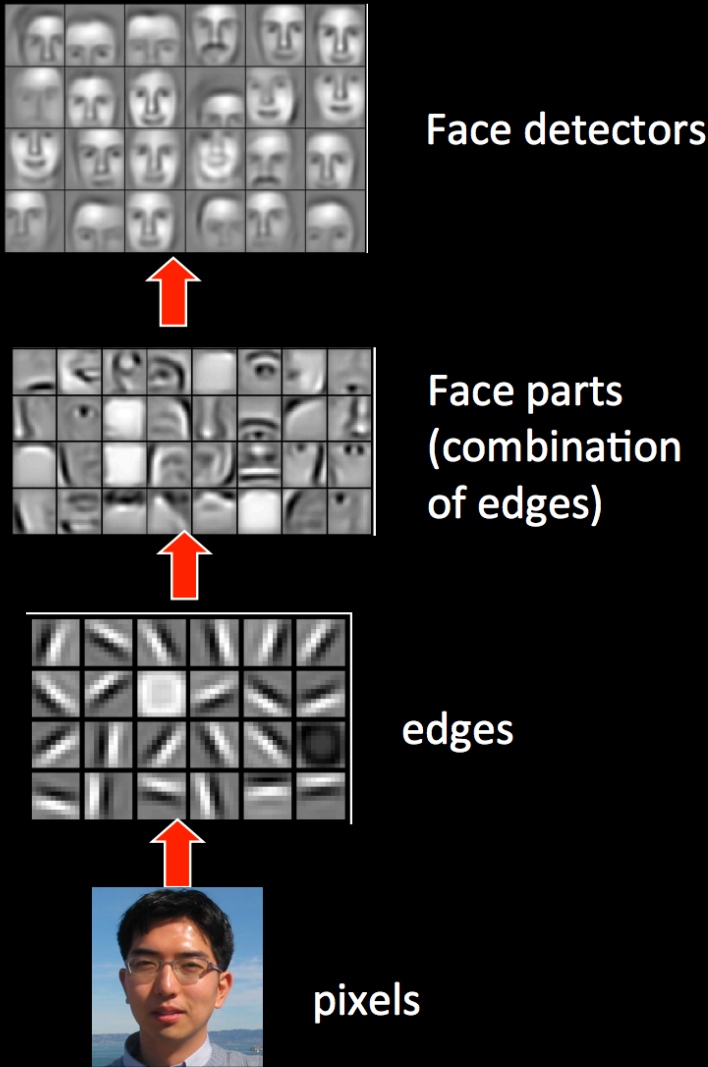
- Each “neuron” might recognize a “part” of a digit.
 - “Deeper” neurons might recognize combinations of parts .
 - Represent complex objects as hierarchical combinations of reusable parts (a simple “grammar”).
- Watch the full video here:
 - <https://www.youtube.com/watch?v=aircAruvnKk>
- Theory:
 - 1 big-enough hidden layer already gives universal approximation.
 - But some functions require exponentially fewer parameters to approximate with more layers (can fight curse of dimensionality).



Hierarchically composed feature representations



Hierarchy of feature representations



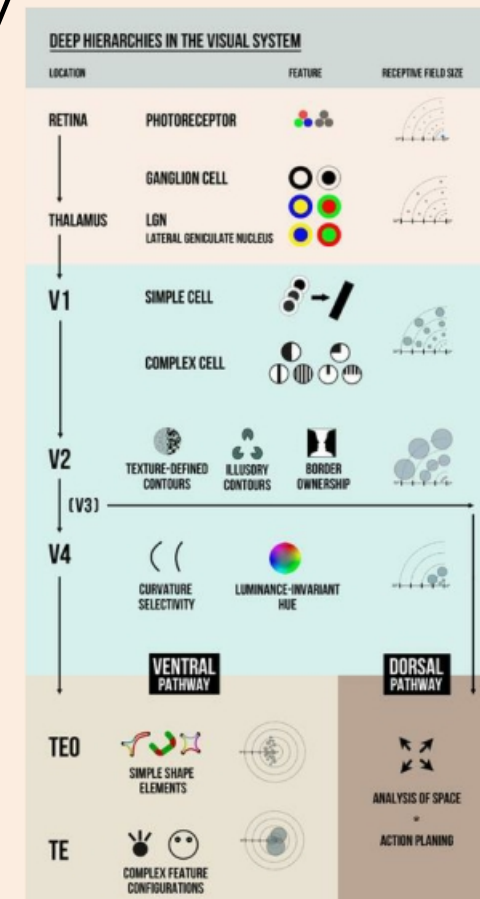
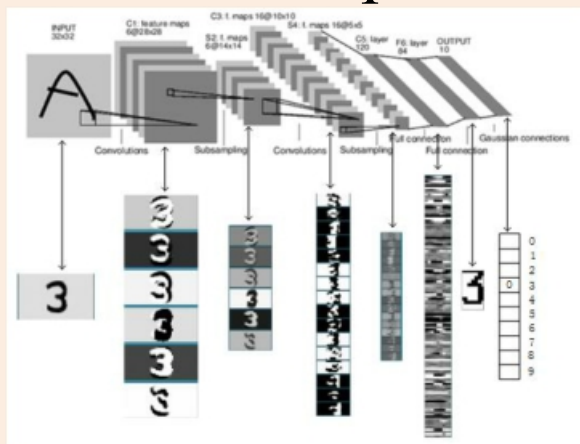
what input makes that
activation Huge!
→ more complex when going
deeper!

Next Topic: Some History about DL

bonus!

ML and Deep Learning History

- 1970 and 1980s: **Connectionism** (brain-inspired ML)
 - Want “connected **networks of simple units**”.
 - Use **parallel computation** and **distributed representations**.
 - **Adding hidden layers z_i** increases expressive power.
 - With 1 layer and enough sigmoid units, a **universal approximator**.
 - Success in optical character recognition.

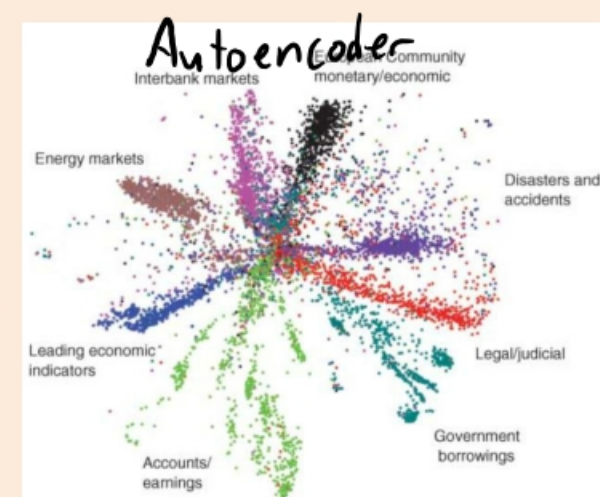
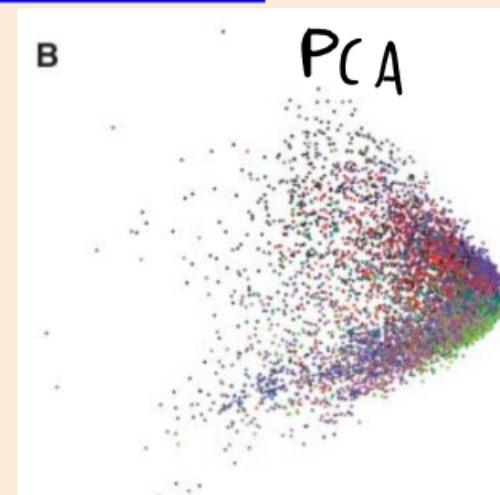
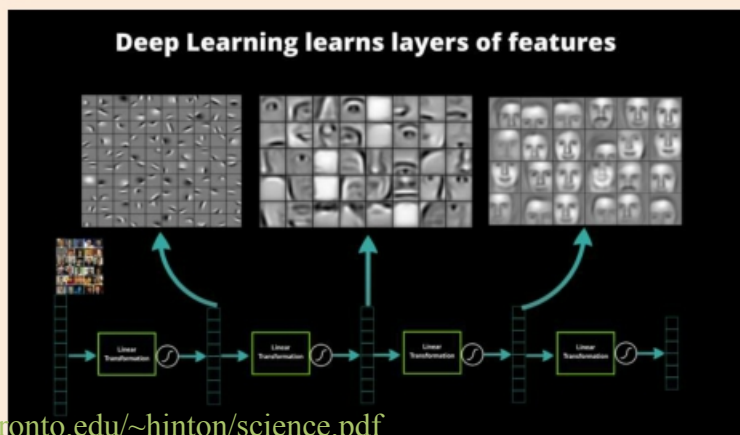


ML and Deep Learning History

- 1990s and early 2000s: drop in popularity.
 - It **proved really difficult to get multilayer models working** robustly.
 - We obtained similar performance with simpler models:
 - Rise in popularity of **logistic regression and SVMs with regularization and kernels** .
 - Lots of internet successes (spam filtering, web search, recommendation).
 - ML moved closer to other fields like numerical optimization and statistics.

ML and Deep Learning History

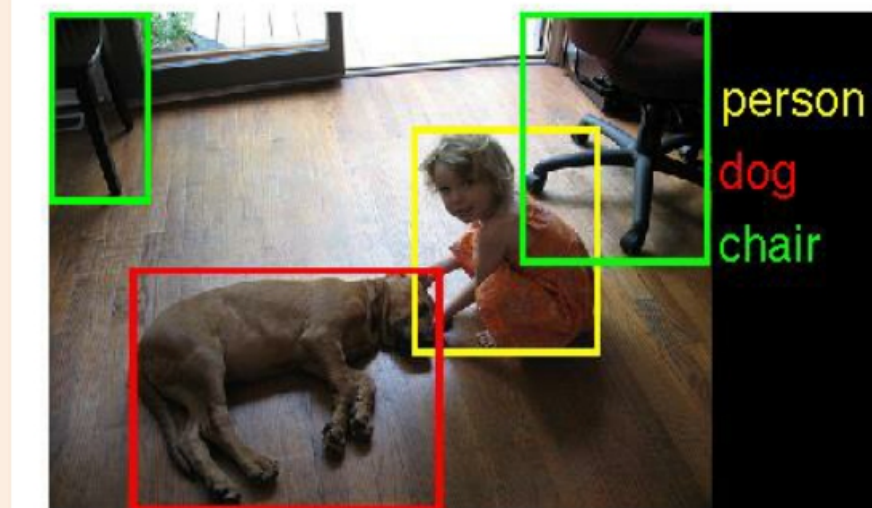
- Late 2000s: push to revive connectionism as “**deep learning**”.
 - Canadian Institute For Advanced Research (CIFAR) NCAP program:
 - “Neural Computation and Adaptive Perception”.
 - Led by Geoff Hinton, Yann LeCun , and Yoshua Bengio
 - Unsupervised successes: “deep belief networks” and “autoencoders”.
 - Could be used to initialize deep neural networks.



bonus!

2010s: DEEP LEARNING!!!

- Bigger datasets, bigger models, parallel computing (GPUs/clusters).
 - And some tweaks to the models from the 1980s.
- Huge improvements in automatic speech recognition (2009).
 - All phones now have deep learning.
- Huge improvements in computer vision (2012).
 - Changed computer vision field almost instantly.
 - Now is how most CV (and AI) is done



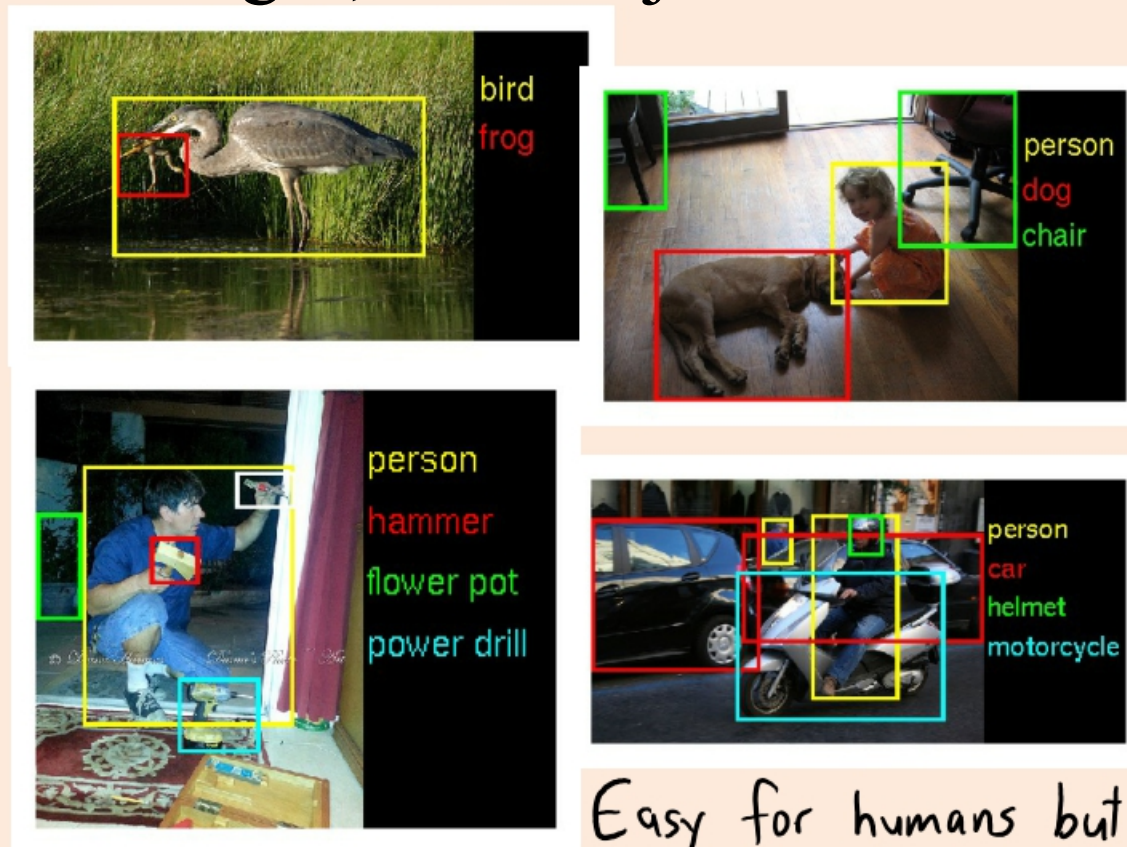
2010s: DEEP LEARNING!!!

- Media hype:
 - “How many computers to identify a cat? 16,000”
New York Times (2012).
 - “Why Facebook is teaching its machines to think like humans”
Wired (2013).
 - “What is ‘deep learning’ and why should businesses care?”
Forbes (2013).
 - “Computer eyesight gets a lot more accurate”
New York Times (2014).
- 2015: huge improvement in language understanding

bonus!

ImageNet Challenge

- Millions of labeled images, 1000 object classes.



Easy for humans but
hard for computers.

ImageNet Challenge

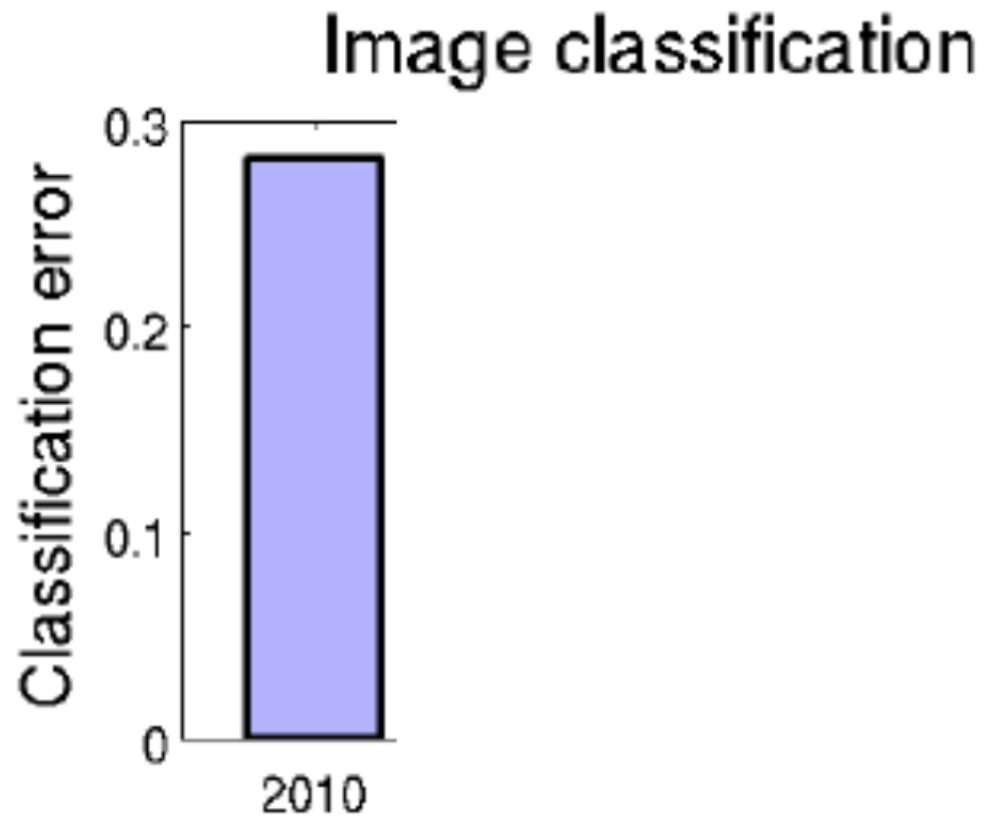
- Object detection task:
 - Single label per image.
 - Humans: $\sim 5\%$ error.



Syberian Husky



Canadian Husky



ImageNet Challenge

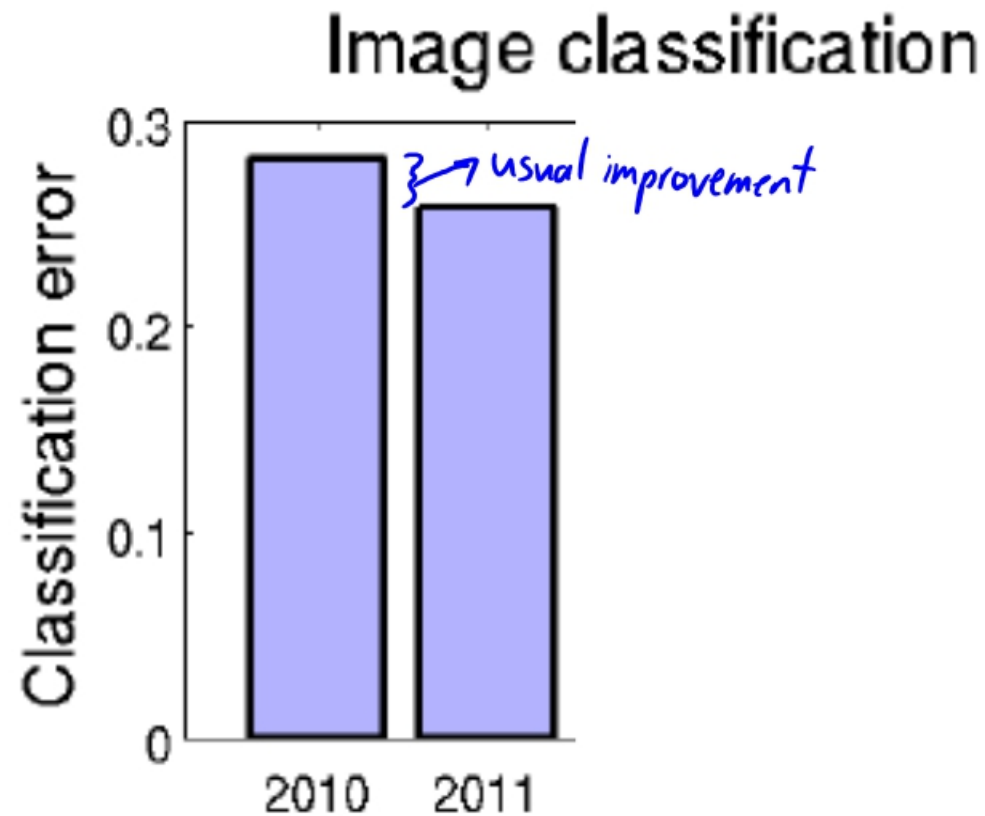
- Object detection task:
 - Single label per image.
 - Humans: $\sim 5\%$ error.



Syberian Husky



Canadian Husky



ImageNet Challenge

- Object detection task:
 - Single label per image.
 - Humans: $\sim 5\%$ error.



Syberian Husky



Canadian Husky



ImageNet Challenge

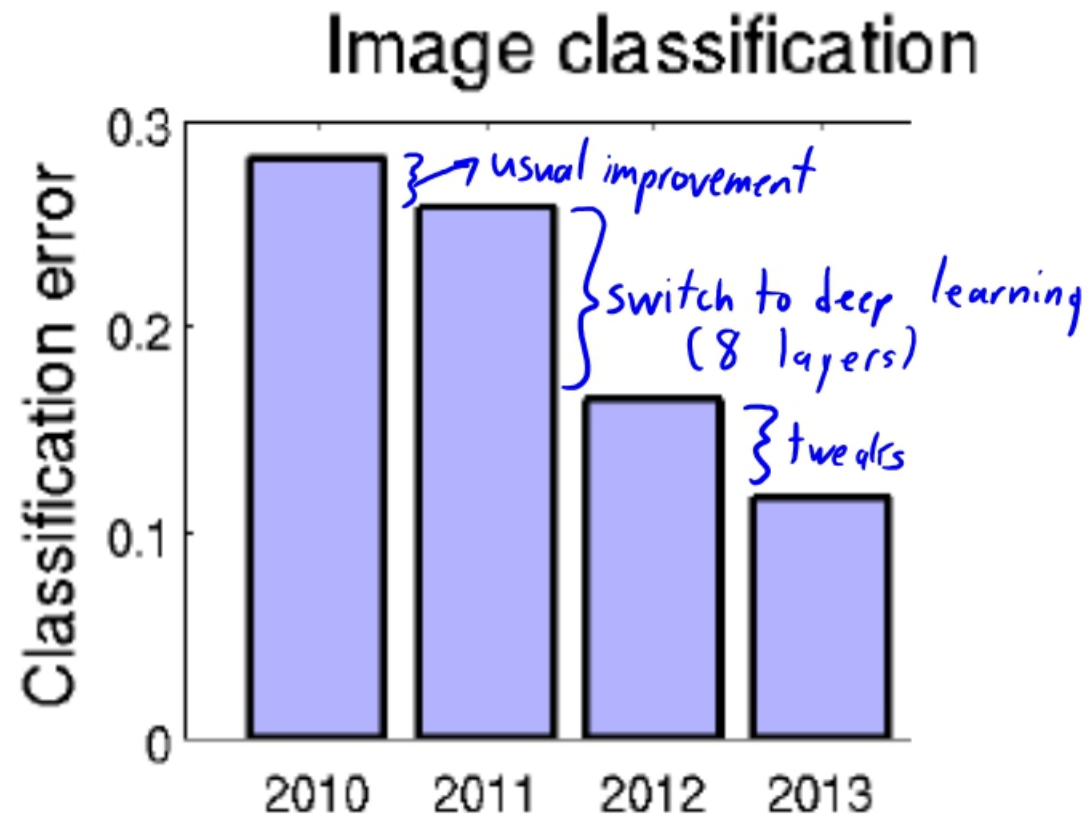
- Object detection task:
 - Single label per image.
 - Humans: $\sim 5\%$ error.



Syberian Husky



Canadian Husky



ImageNet Challenge

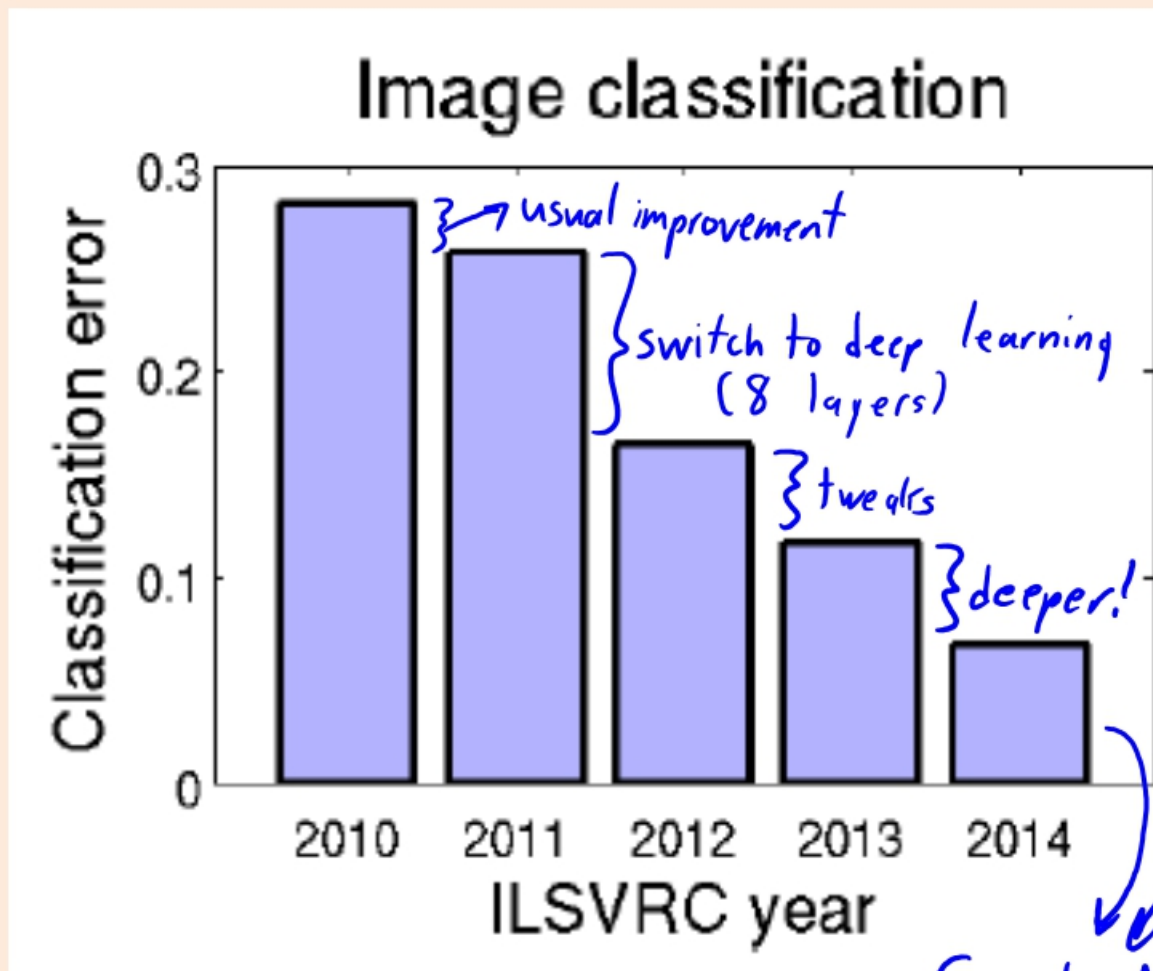
- Object detection task:
 - Single label per image.
 - Humans: ~5% error.



Syberian Husky



Canadian Husky

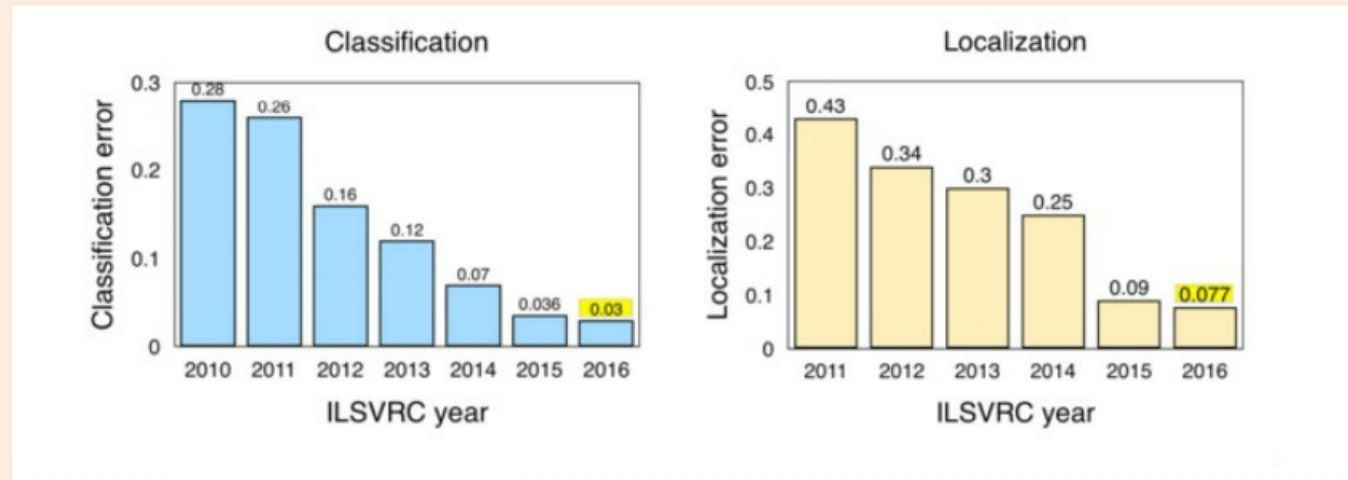


GoogleNet:
6.7% error
22 layers



ImageNet Challenge

- Object detection task:
 - Single label per image.
 - Humans: $\sim 5\%$ error.
- 2015: Won by Microsoft Asia
 - 3.6% error.
 - 152 layers, introduced “ResNets”.
 - Also won “localization” (finding location of objects in images).
- 2016: Chinese University of Hong Kong:
 - Ensembles of previous winners and other existing methods.
- 2017: fewer entries, organizers decided this would be last year.



bonus!

Image Classification on ImageNet

Leaderboard

Dataset

View

Top 1 Accuracy

▼

by

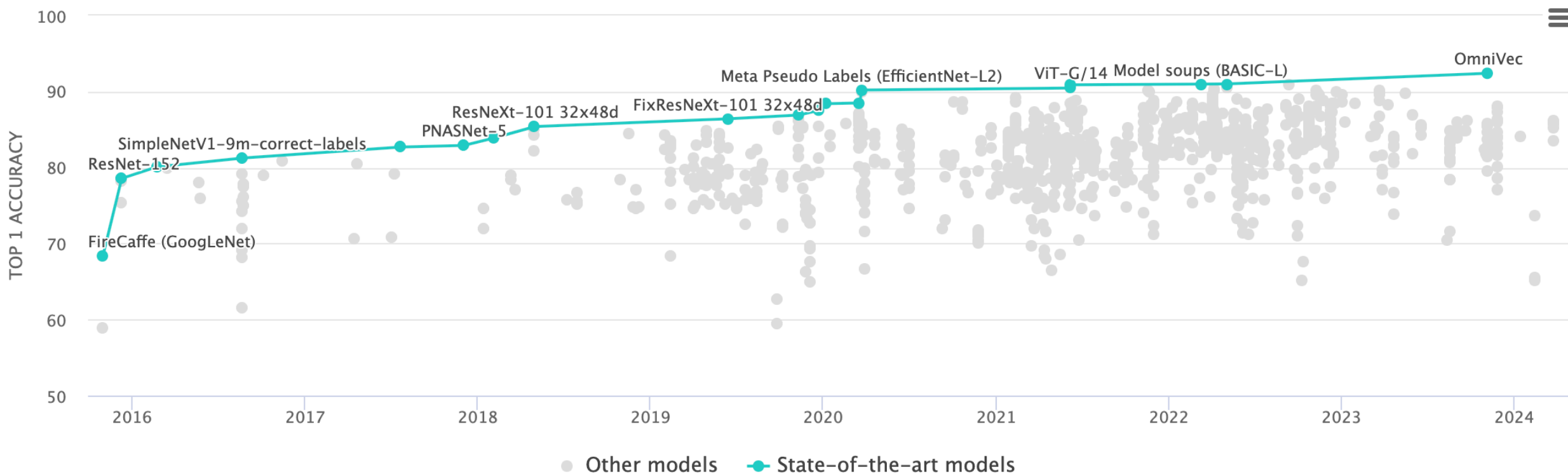
Date

▼

for

All models

▼



Summary

- Deep learning:
 - Neural networks with multiple hidden layers.
- Intuition about neural networks solving the XOR problem.
- Biological motivation for deep neural networks.
 - Can more efficiently represent some functions
 - Unprecedented performance on difficult pattern recognition tasks.
- Some history about deep neural networks.

Multiple Word Prototypes

- What about **homonyms** and **polysemy**?
 - The word vectors would **need to account for all meanings**.
- More recent approaches:
 - Try to **cluster the different contexts** where words appear.
 - Use **different vectors for different contexts**.

$$X_{jaguar} \approx \begin{bmatrix} \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \end{bmatrix} \begin{matrix} z_{j1} \\ z_{j2} \\ z_{j3} \end{matrix}$$

